Pandas : limites et écosystèmes alternatifs

Lino Galiana
Version du 20 octobre 2026

Après avoir exploré en profondeur l'écosystème `Pandas`, ce court chapitre prend du recul sur quelques limites de sa syntaxe et présente les principaux paradigmes alternatifs (`Polars`, `DuckDB`, `Spark`) qui permettent d'aller au-delà, notamment sur de gros volumes de données.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **1**
1.1 Données de ce chapitre 2
2 `Pandas` dans une chaine d'opérations **2**
2.1 Quelques limites sur la syntaxe de `Pandas` 3
3 Les autres paradigmes **5**
3.1 `Polars` 5
3.2 `DuckDB` 6
3.3 `Spark` et le *big data* 7
Informations additionnelles 7
Bibliographie **9**

Compétences à l'issue de ce chapitre

- Connaître les principales limites de la syntaxe de `Pandas` ;
- Savoir chaîner des opérations de manière plus lisible grâce aux *pipes* ;
- Connaître les principaux *packages* alternatifs à `Pandas` (`Polars`, `DuckDB`, `Spark`).

1 Introduction

Les deux chapitres précédents ont permis d'explorer en profondeur la richesse de l'écosystème `Pandas`, qu'il s'agisse d'associer des données grâce à des jointures ou de construire des statistiques descriptives et de restructurer des données. `Pandas` est un indispensable dans la boite à outil du *data scientist*.

Ce court chapitre propose de prendre un peu de recul. Malgré toutes ses qualités, `Pandas` n'est pas parfait: sa syntaxe, héritée de plus de quinze ans d'histoire, comporte quelques aspérités. Nous verrons également que d'autres paradigmes, plus récents, permettent d'aller au-delà de `Pandas`, notamment lorsqu'il s'agit de traiter des volumes de données plus importants.

1.1 Données de ce chapitre

Pour illustrer ce chapitre, nous repartons du jeu de données brut des émissions de gaz à effet de serre de l'Ademe, déjà utilisé dans les chapitres précédents:

```
import numpy as np
import pandas as pd

url   =   "https://data.ademe.fr/data-fair/api/v1/datasets/igt-pouvoir-de-rechauffement-global/
convert"
emissions = pd.read_csv(url)
emissions['dep'] = emissions["INSEE commune"].str[:2]
```

Pour obtenir des résultats reproductibles, on peut fixer la racine du générateur pseudo-aléatoire.

```
import numpy as np
np.random.seed(123)
```

2 `Pandas` dans une chaine d'opérations

En général, dans un projet, le nettoyage de données va consister en un ensemble de méthodes appliquées à un `DataFrame` ou alors une `Serie` lorsqu'on travaille exclusivement sur une colonne. Autrement dit, ce qui est généralement attendu lorsqu'on fait du `Pandas` c'est d'avoir une chaîne qui prend un `DataFrame` en entrée et ressort ce même `DataFrame` enrichi, ou une version agrégée de celui-ci, en sortie.

Cette manière de procéder est le coeur de la syntaxe `dplyr` en `R` mais n'est pas forcément native en `Pandas` selon les opérations qu'on désire mettre en oeuvre. En effet, la manière naturelle de mettre à jour un *dataframe* en `Pandas` passe souvent par une syntaxe du type:

```
import numpy as np
import pandas as pd

data = [[8000, 1000], [9500, np.nan], [5000, 2000]]
df = pd.DataFrame(data, columns=['salaire', 'autre_info'])
df['salaire_net'] = df['salaire']*0.8
```

En `SQL` on pourrait directement mettre à jour notre base de données avec la nouvelle colonne :

```
SELECT *, salaire*0.8 AS salaire_net FROM df
```

L'écosystème du *tidyverse* en `R`, l'équivalent de `Pandas`, fonctionne selon la même logique que SQL de mise à jour de table. On ferait en effet la commande suivante avec `dplyr`:

```
df %>% mutate(salaire_net = salaire*0.8)
```

Techniquement on pourrait faire ceci avec un `assign` en `Pandas`

```
df = df.drop("salaire_net", axis = "columns")
df = df.assign(salaire_net = lambda s: s['salaire']*0.8)
```

Ligne 1
Pour effacer la variable afin de repartir de l'exemple initial

Cependant cette syntaxe `assign` n'est pas très naturelle. Il est nécessaire de lui passer une *lambda function* qui attend comme *input* un `DataFrame` là où on voudrait une colonne. Il ne s'agit donc pas vraiment d'une syntaxe lisible et pratique.

Il est néanmoins possible d'enchaîner des opérations sur des jeux de données grâce aux *pipes*. Ceux-ci reprennent la même philosophie que celle de `dplyr`, elle-même inspirée du *pipe* Linux. Cette approche permettra de rendre plus lisible le code en définissant des fonctions effectuant des opérations sur une ou plusieurs colonnes d'un DataFrame. Le premier argument à indiquer à la fonction est le `DataFrame`, les autres sont ceux permettant de contrôler son comportement

```
def calcul_salaire_net(df: pd.DataFrame, col: str, taux: float = 0.8):
  df["salaire_net"] = df[col]*taux
  return df
```

Ce qui transforme notre chaine de production en

```
(
  df
  .pipe(calcul_salaire_net, "salaire")
)
```

	salaire	autre_info	salaire_net
0	8000	1000.0	6400.0
1	9500	NaN	7600.0
2	5000	2000.0	4000.0

2.1 Quelques limites sur la syntaxe de `Pandas`

Il y a un avant et un après `Pandas` dans l'analyse de données en `Python`. Sans ce *package* ô combien pratique `Python`, malgré toutes les forces de ce langage, aurait eu du mal à s'installer dans le paysage de l'analyse de données. Cependant, si `Pandas` propose une syntaxe cohérente sur de nombreux aspects, elle n'est pas parfaite non plus. Les paradigmes plus récents d'analyse de données en `Python` ont d'ailleurs parfois l'ambition de corriger ces imperfections syntaxiques là.

Parmi les points les plus génants au quoditien il y a le besoin de régulièrement faire des `reset_index` lorsqu'on construit des statistiques descriptives. En effet, il peut être dangereux de garder des indices qu'on ne contrôle pas bien car, sans attention de notre part lors des phases de *merge*, ils peuvent être utilisés à mauvais escient par `Pandas` pour joindre les données ce qui peut provoquer des suprises.

`Pandas` est extrêmement bien fait pour restructurer des données du format *long* to *wide* ou *wide* to *long*. Cependant, ce n'est pas la seule manière de restructurer un jeu de données qu'on peut vouloir mettre en oeuvre. Il arrive régulièrement qu'on désire comparer la valeur d'une observation à celle d'un groupe à laquelle elle appartient. C'est notamment particulièrement utile dans une phase d'analyse des anomalies, valeurs aberrantes ou lors d'une investigation de détection de fraude. De manière native, en `Pandas`, il faut construire une statistique agrégée par groupe et refaire un *merge* aux données initiales par le biais de la variable de groupe. C'est un petit peu fastidieux:

```
emissions_moyennes = emissions.groupby("dep").agg({"Agriculture": "mean"}).reset_index()
emissions_enrichies = (
  emissions
  .merge(emissions_moyennes, on = "dep", suffixes = ['', '_moyenne_dep'])
)
emissions_enrichies['relatives']                                                    =
```

```
emissions_enrichies["Agriculture"]/emissions_enrichies["Agriculture_moyenne_dep"]
emissions_enrichies.head()
```

	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep	Agriculture_moyenne_dep	relatives
0	01001	L'ABERGEMENT-CLEMENCIAT	3711.425991	NaN	NaN	432.751835	101.430476	2.354558	6.911213	309.358195	793.156501	367.036172	01	1974.535382	1.879645
1	01002	L'ABERGEMENT-DE-VAREY	475.330205	NaN	NaN	140.741660	140.675439	2.354558	6.911213	104.866444	348.997893	112.934207	01	1974.535382	0.240730
2	01004	AMBERIEU-EN-BUGEY	499.043526	212.577908	NaN	10313.446515	5314.314445	998.332482	2930.354461	16616.822534	15642.420313	10732.376934	01	1974.535382	0.252740
3	01005	AMBERIEUX-EN-DOMBES	1859.160954	NaN	NaN	1144.429311	216.217508	94.182310	276.448534	663.683146	1756.341319	782.404357	01	1974.535382	0.941569
4	01006	AMBLEON	448.966808	NaN	NaN	77.033834	48.401549	NaN	NaN	43.714019	398.786800	51.681756	01	1974.535382	0.227378

Dans le *tidyverse*, cette opération en deux temps pourrait être faite en une seule étape, ce qui est plus pratique

```
emissions %>%
  group_by(dep) %>%
  mutate(relatives = Agriculture/mean(Agriculture))
```

Ce n'est pas si grave mais cela alourdit la longueur des chaines de traitement faites en `Pandas` et donc la charge de maintenance pour les faire durer dans le temps.

De manière plus générale, les chaînes de traitement `Pandas` peuvent être assez verbeuses, car il faut régulièrement redéfinir le `DataFrame` qu'on utilise plutôt que simplement les colonnes. Par exemple, pour faire un filtre sur les lignes et les colonnes, il faudra faire:

```
(
  emissions
  .loc[
    (emissions["dep"] == "12") & (emissions["Routier"]>500), ['INSEE commune', 'Commune']
  ]
  .head(5)
)
```

	INSEE commune	Commune
4058	12001	AGEN-D'AVEYRON
4059	12002	AGUESSAC
4062	12006	ALRANCE
4063	12007	AMBEYRAC
4064	12008	ANGLARS-SAINT-FELIX

En SQL on pourrait se contenter de faire référence aux colonnes dans le filter

```
SELECT "INSEE commune", 'Commune'
FROM emissions
WHERE dep=="12" AND Routier>500
```

Dans le *tidyverse* (`R`) on pourrait aussi faire ceci simplement

```
df %>%
  filter(dep=="12", Routier>500) %>%
  select(`INSEE commune`, `Commune`)
```

3 Les autres paradigmes

Malgré toutes les limites que nous venons d'évoquer, et les solutions alternatives que nous allons présenter, `Pandas` reste LE *package* central de l'écosystème de la donnée avec `Python`. Nous allons voir dans les prochains chapitres son intégration native à l'écosystème `Scikit` pour le *machine learning* ou l'extension de `Pandas` aux données spatiales avec `GeoPandas`.

Les autres solutions techniques que nous allons ici évoquer peuvent être pertinentes si on désire traiter des volumes de données importants ou si on désire utiliser des syntaxes alternatives.

Les principales alternatives à `Pandas` sont `Polars`, `DuckDB` et `Spark`. Il existe également `Dask`, une librairie pour paralléliser des traitements écris en `Pandas`.

3.1 `Polars`

`Polars` est certainement le paradigme le plus inspiré de `Pandas`, jusqu'au choix du nom. La première différence fondamentale est dans les couches internes utilisées. `Polars` s'appuie sur l'implémentation `Rust` de `Arrow` là où `Pandas` s'appuie sur `Numpy` ce qui est facteur de perte de performance. Cela permet à `Polars` d'être plus efficace sur de gros volumes de données, d'autant que de nombreuses opérations sont parallélisées et reposent sur l'évaluation différées (*lazy evaluation*) un principe de programmation qui permet d'optimiser les requêtes pour ne pas les exécuter dans l'ordre de définition mais dans un ordre logique plus optimal.

Une autre force de `Polars` est la syntaxe plus cohérente, qui bénéficie du recul d'une quinzaine d'années d'existence de `Pandas` et d'une petite dizaine d'années de `dplyr` (le *package* de manipulation de données au sein du paradigme du *tidyverse* en `R`). Pour reprendre l'exemple précédent, il n'est plus nécessaire de forcer la référence au *DataFrame*, dans une chaîne d'exécution toutes les références ultérieures seront faites au regard du *DataFrame* de départ

```
import polars as pl
emissions_polars = pl.from_pandas(emissions)
(
  emissions_polars
  .filter(pl.col("dep") == "12", pl.col("Routier") > 500)
  .select('INSEE commune', 'Commune')
  .head(5)
)
```

INSEE commune	Commune
str	str
"12001"	"AGEN-D'AVEYRON"
"12002"	"AGUESSAC"
"12006"	"ALRANCE"
"12007"	"AMBEYRAC"
"12008"	"ANGLARS-SAINT-FELIX"

Pour découvrir `Polars`, de nombreuses ressources en ligne sont accessibles, notamment ce *notebook* construit pour le réseau des *data scientists* de la statistique publique.

3.2 DuckDB

DuckDB est le nouveau venu dans l'écosystème de l'analyse de données repoussant les limites des données pouvant être traitées avec `Python` sans passer par des outils *big data* comme `Spark`. DuckDB est la quintessence d'un nouveau paradigme, celui du *« Big data is dead »*, où on peut traiter des données de volumétrie importante sans recourir à des infrastructures imposantes.

Outre sa grande efficacité, puisqu'avec DuckDB on peut traiter des données d'une volumétrie supérieure à la mémoire vive de l'ordinateur ou du serveur, DuckDB présente l'avantage de proposer une syntaxe uniforme quelle que soit le langage qui appelle DuckDB (`Python`, `R`, `C++` ou `Javascript`). DuckDB privilégie la syntaxe SQL pour traiter les données avec de nombreuses fonctions pré-implementées pour simplifier certaines transformations de données (par exemple pour les données textuelles, les données temporelles, etc.).

Par rapport à d'autres systèmes s'appuyant sur SQL, comme `PostGreSQL`, `DuckDB` est très simple d'installation, ce n'est qu'une librairie `Python` là où beaucoup d'outils comme `PostGreSQL` nécessite une infrastructure adaptée.

Pour reprendre l'exemple précédent, on peut utiliser directement le code SQL précédent

```
import duckdb
duckdb.sql(
  """
  SELECT "INSEE commune", "Commune"
  FROM emissions
  WHERE dep=='12' AND Routier>500
  LIMIT 5
  """)
```

```
┌──────────────┬──────────────────┐
│ INSEE commune │     Commune      │
│    varchar    │     varchar      │
├──────────────┼──────────────────┤
│ 12001         │ AGEN-D'AVEYRON   │
│ 12002         │ AGUESSAC         │
│ 12006         │ ALRANCE          │
│ 12007         │ AMBEYRAC         │
│ 12008         │ ANGLARS-SAINT-FELIX │
└──────────────┴──────────────────┘
```

Ici la clause `FROM emissions` vient du fait qu'on peut directement exécuter du SQL depuis un objet `Pandas` par le biais de `DuckDB`. Si on fait la lecture directement dans la requête, celle-ci se complexifie un petit peu mais la logique est la même

```
import duckdb
duckdb.sql(
  f"""
  SELECT "INSEE commune", "Commune"
  FROM read_csv_auto("{url}")
  WHERE
    substring("INSEE commune",1,2)=='12'
    AND
    Routier>500
  LIMIT 5
  """)
```

```
┌──────────────┬──────────────────┐
│ INSEE commune │     Commune      │
│    varchar    │     varchar      │
├──────────────┼──────────────────┤
│ 12001        │ AGEN-D'AVEYRON   │
│ 12002        │ AGUESSAC         │
│ 12006        │ ALRANCE          │
│ 12007        │ AMBEYRAC         │
│ 12008        │ ANGLARS-SAINT-FELIX │
└──────────────┴──────────────────┘
```

Le rendu du *DataFrame* est légèrement différent de `Pandas` car, comme `Polars` et de nombreux systèmes de traitement de données volumineuses, `DuckDB` repose sur l'évaluation différée et donc ne présente en *display* qu'un échantillon de données. `DuckDB` et `Polars` sont d'ailleurs très bien intégrés l'un à l'autre. On peut très bien faire du SQL sur un objet `Polars` via `DuckDB` ou appliquer des fonctions `Polars` sur un objet initialement lu avec `DuckDB`.

L'un des intérêts de `DuckDB` est son excellente intégration avec l'écosystème `Parquet`, le format de données déjà mentionné qui devient un standard dans le partage de données (il s'agit, par exemple, de la pierre angulaire du partage de données sur la plateforme *HuggingFace*). Pour en savoir plus sur `DuckDB` et découvrir son intérêt pour lire les données du recensement de la population française, vous pouvez consulter ce post de blog.

3.3 `Spark` et le *big data*

`DuckDB` a repoussé les frontières du *big data* qu'on peut définir comme le volume de données à partir duquel on ne peut plus traiter celles-ci sur une machine sans mettre en oeuvre une stratégie de parallélisation.

Néanmoins, pour les données très volumineuses, `Python` est très bien armé grâce à la librairie `PySpark`. Celle-ci est une API en Python pour le langage `Spark`, un langage *big data* basé sur Scala. Ce paradigme est construit sur l'idée que les utilisateurs de `Python` y accèdent par le biais de *cluster* avec de nombreux noeuds pour traiter la donnée de manière parallèle. Celle-ci sera lue par blocs, qui seront traités en parallèle en fonction du nombre de noeuds parallèles. L'API DataFrame de `Spark` présente une syntaxe proche de celle des paradigmes précédents avec une ingénieurie plus complexe en arrière-plan liée à la parallélisation native.

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
```

```
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "py7zr≥1.1.3",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
    "python-dotenv≥1.0.1",
    "python-frontmatter≥1.1.0",
    "pywaffle≥1.1.1",
    "requests≥2.32.3",
    "scikit-image≥0.24.0",
    "scikit-learn≥1.8.0",
    "scipy≥1.13.0",
    "seaborn≥0.13.2",
    "selenium<4.39.0",
    "spacy≥3.8.4",
    "webdriver-manager≥4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
```

```
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

Bibliographie